Technical Assessment

Design and implement a minimal private video streaming service.

Arsen Korpetayev

Summary

Key Metric	Proposed solution
Support common video formats	Video Transcoding
Time-to-stream	1. Segmenting video for parallel transcoding 2. Prioritizing low resolution transcoding, then upgrading to high resolution
Consistent playback performance	1. HTTP Live Streaming 2. CDN 3. Automatic resolution based on bandwidth
Horizontally scalable	1. Stateless servers 2. NoSQL database (DynamoDB)
Cost efficient	1. Cloudflare CDN for no egress fees 2. Presigned URLs for clients to upload straight to storage avoiding servers 3. Short storage policy for originals in R2

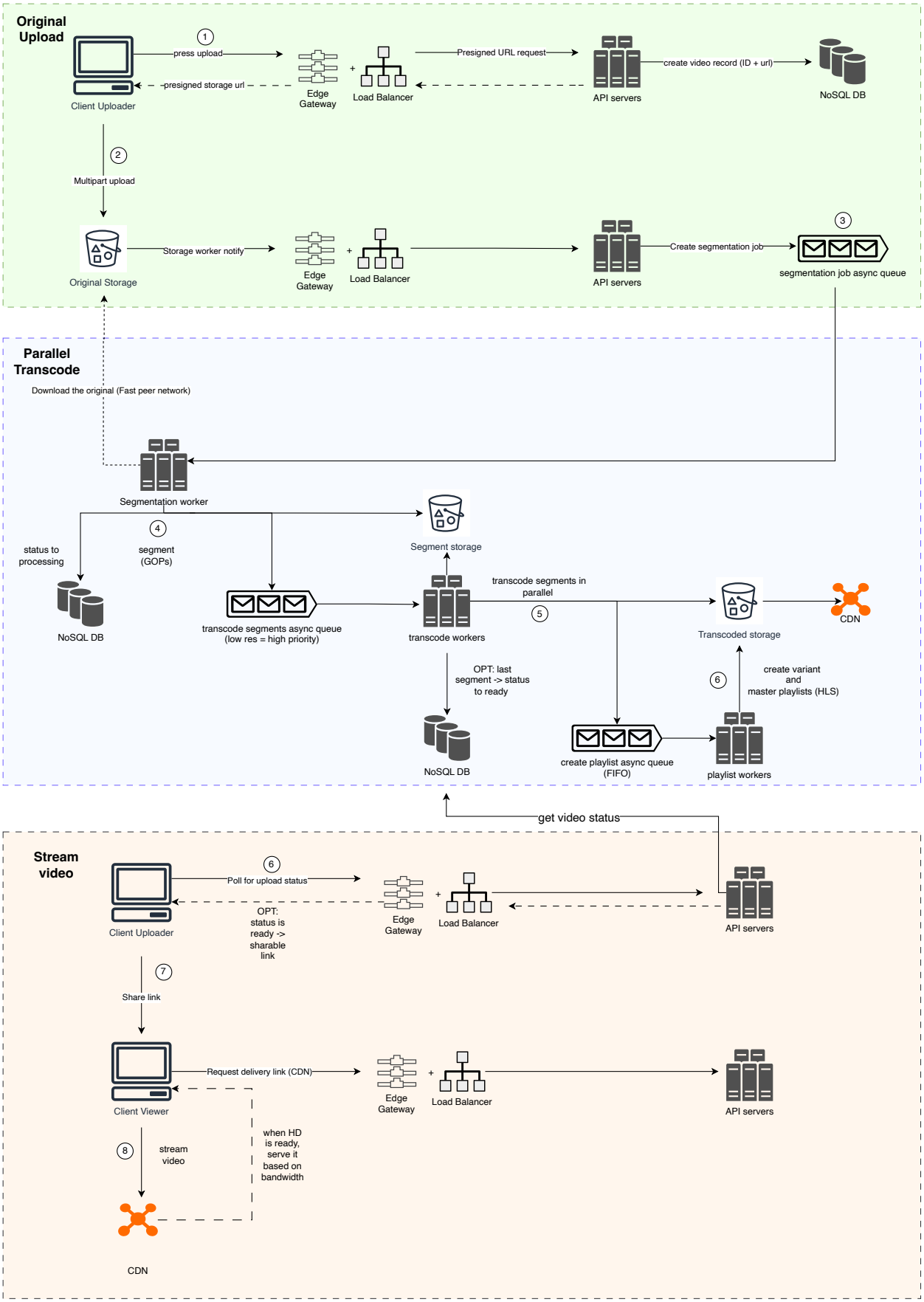
Cost estimation

Assuming a baseline scale of 100,000 DAU (yielding 150,000 uploaded minutes and 10TB of delivery egress per month), this model relies on commodity compute and Cloudflare's zero-egress pricing. The comparison below utilizes published baseline pricing: AWS MediaConvert Basic tier (~$0.045/min for a standard 4-tier ABR ladder) plus CloudFront egress ($0.085/GB), contrasted against four commodity 8-core compute nodes running our Rust workers (e.g., Hetzner CCX33 at ~$45/mo) and Cloudflare R2's zero-cost egress model.

Architecture	Transcode Compute	Delivery Egress (10TB)	Est. Monthly Cost
AWS Managed (MediaConvert + CloudFront)	~$6,750	~$850	**~$7,600**
This Design (Commodity Rust Workers + R2)	~$180 (4x 8-core nodes)	$0	**~$180**

Result: ~97% cost reduction, allowing uninhibited horizontal scaling.

High Level Architecture Overview



Original Upload Phase
This phase handles the initial ingestion of the video file from the user.

Initialization: The Client Uploader initiates the process by pressing upload. The request passes through an Edge Gateway and a Load Balancer to reach the API servers.

Database Record & URL: The API servers create a new video record (containing an ID and URL) in a NoSQL DB and request a presigned URL. A presigned storage URL is then returned to the client.

File Transfer: The client performs a multipart upload directly into the Original Storage using the provided URL. Avoiding slow client upload to transcoding server saves CPU time and prevents worker hanging. The original video is later downloaded for transcoding by a fast Cloudflare-to-AWS peer network

Job Creation: Once the upload finishes, a storage worker notification triggers the API servers to create a "segmentation job," which is placed into a segmentation job async queue.

Parallel Transcode Phase
This phase prepares the video for streaming by breaking it into smaller pieces and converting it into different formats.

Segmentation: A Segmentation worker retrieves the job from the queue and downloads the original file over a fast peer network. It immediately updates the video's status in the NoSQL DB to "processing".

Chunking: The worker uses ffmpeg to slice the video into small segments, known as GOPs (Group of Pictures).

Queuing for Transcode: These raw segments are placed into R2 storage and transcode segment jobs are added to the async queue. The queue prioritizes low-resolution segments to ensure the video becomes playable as quickly as possible.

Parallel Processing: Multiple Transcode workers pull segments from this queue and transcode them in parallel.

Storage & CDN Routing: The processed segments are saved to Transcoded storage and subsequently pushed to a Content Delivery Network (CDN). Also, playlist creation jobs are added to the queue

Playlist Creation: Workers pick up the playlist creation jobs sequentially (FIFO with MessageGroupId = video_id) which prevents race conditions. Very fast db query to check ready resolutions and quick text doc creation. Does not block for more than 50ms. updates the overall video status in the NoSQL DB to "ready".

Stream Video Phase
This phase handles the delivery of the processed video to end-users.

Status Polling: The original Client Uploader periodically polls the system for the upload status via the API servers.

Link Generation: Once the status is marked as "ready," the system provides a shareable link.

Sharing: The uploader shares this link with a Client Viewer.

Playback Request: The Client Viewer requests a delivery link from the API servers, which directs them to the CDN.

Streaming: The viewer streams the video directly from the CDN. The system is designed to dynamically serve high-definition (HD) segments based on the user's available bandwidth once those specific segments are ready.

Database

NoSQL databases are designed with native support for sharding/parallelization, usually using consistent hashing and some form of hash rings. While relational databases can be sharded, the process is massively complex and goes against the "minimal" and "maintainable" requirements of this assessment. Additionally, this streaming service being private means complex queries granted by SQL are not a priority

CAP theorem states that either consistency or availability can be optimised for in a sharded database. Our system requires strong consistency due to the parallel transcoding step. This means that solutions such as Cassandra, which focus on availability and only offer eventual consistency, are not suitable. DynamoDB offers Atomic Updates which allows our parallel transcoding workers to keep track of total processed segments.

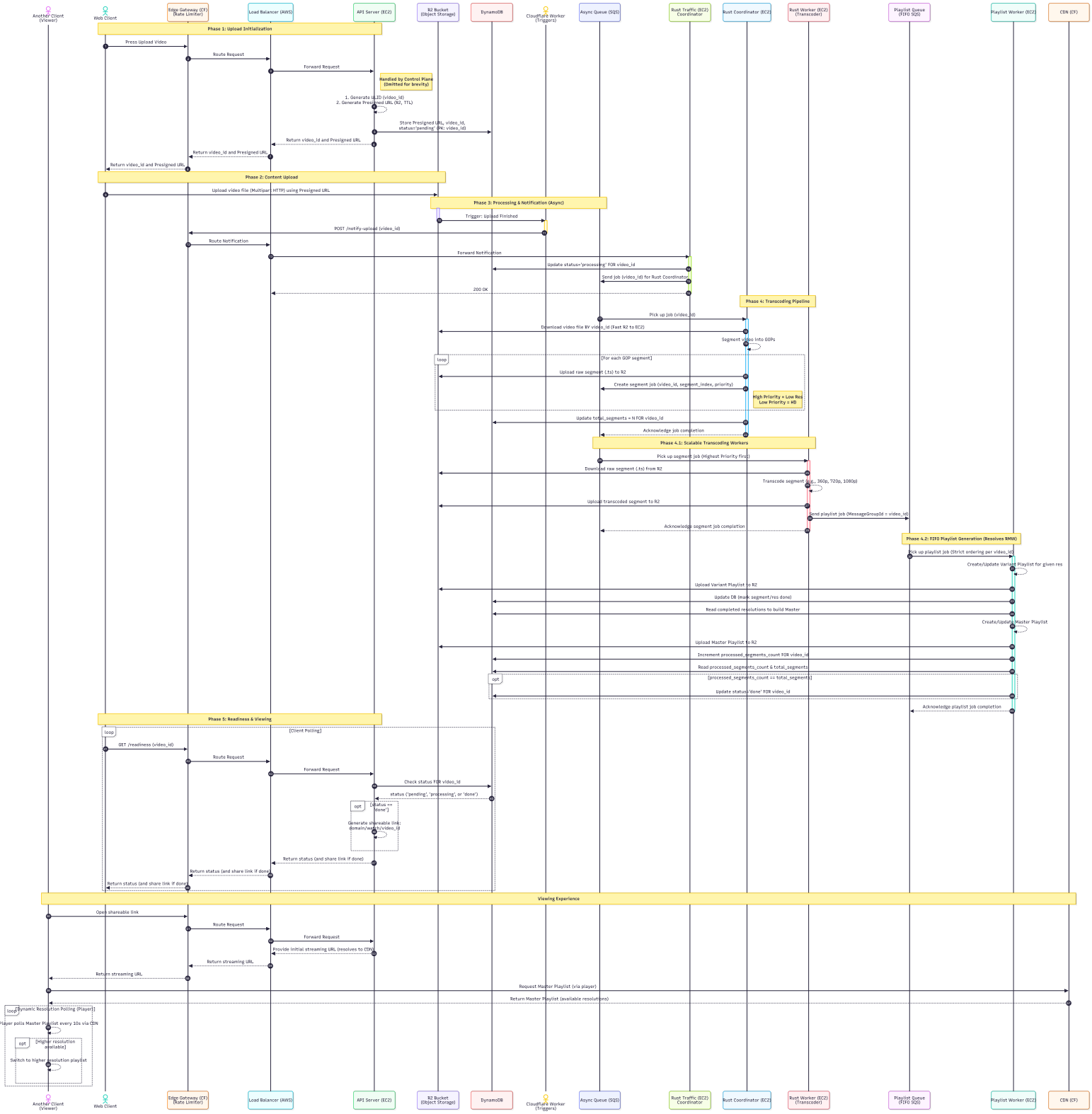
Thus, DynamoDB was chosen.
ULID (Universally Unique Lexicographically Sortable Identifier) is used a partition key for even partitioning, time sorting (in search engines), cost efficiency (shorter string than UUID) and ease of implementation in a distributed system (as opposed to Snowflake). The object model:

```
{
  "video_id": "1543818318210334 72",
  "presigned_url": "https://our-bucket.r2.region.cloudflare./...",
  "status": "pending",
  "total_raw_segments": 12,
  "processed_{resolution}": 0,
  "author_name": "TechReviewer99"
}
```

For anonymous submission author_name is simply omitted. Because every partition holds exactly one record, sort key is not used. Additionally, as the service is private, querying by author_name for example is not in the scope. However, if queries are required in the future, the database can stream to a dedicated search engine like Elasticsearch, Amazon OpenSearch, or Algolia, upon a status == "ready" trigger. In such case, queries are automatically time-sorted due to ULID leading with a timestamp.

Continued on the next page

Detailed Sequence



The High Level Overview section glossed over segment upload to R2 and HTTP Live Streaming, which is illustrated in the detailed sequence diagram:
When playlist workers create a variant playlist, which lists all the segments comprising the video and their timestamps:

```
#EXTM3U
#EXT-X-TARGETDURATION:10
#EXTINF:2.100, chunk_0.ts
#EXTINF:9.050, chunk_1.ts
#EXTINF:1.500, chunk_2.ts
```

these timestamps are used for consistent playback in HTTP Live Streaming.
Additionally, a master playlist is created which lists all the resolutions and their associated required bandwidth to stream.
While lower resolution will be available first (due to internal priority and faster compute time), the client code will implement a "soft reload" by running a background JavaScript interval that independently fetches the master playlist.

Steps for local testing

The provided monorepo includes both the Rust cargo workspace backend and Svelte frontend. The code uses AWS SDK to let EC2 hosted workers and API servers communicate with R2, AWS SQS and DynamoDB. Because LocalStack is set up in the docker-compose.yml to provide local emulations of S3 (R2 compatible), DynamoDB and SQS, no code changes are required to test locally vs deploying to cloud, it is only a matter of url and address configuration. The only prerequisite for local testing is having Docker installed and running
To test locally:

1. `docker compose up --build`
2. Once console shows "LocalStack initialization complete!" Navigate to: http://localhost:8080/ in your browser

Code structure is detailed in README.md

Thank you for your time!